

MODUL PRAKTIKUM PERTEMUAN 1

Persiapan Lingkungan & Sintaks Go dengan Fiber

Praktikum Pemrograman Backend Lanjut

Komponen	Keterangan
Alokasi waktu	100 menit sesi laboratorium + 100 menit kerja mandiri
Luaran	Lingkungan kerja siap pakai dan satu REST API sederhana berbasis Fiber yang berjalan
Bentuk tugas	PDF laporan + repositori GitHub

Tujuan Pembelajaran

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Menyiapkan lingkungan pengembangan Go beserta perkakas pendukungnya
2. Mendeklarasikan dan menginisialisasi variabel serta konstanta dalam Go
3. Menggunakan struktur data dasar Go: array, slice, map, dan channel
4. Menjelaskan konsep pointer dan perbedaan pass by value dengan pass by reference
5. Membuat struct beserta method-nya dan memakainya dalam aplikasi web

A. Landasan Teori

1. Posisi Pertemuan Ini

Anda sudah pernah membangun REST API dengan bahasa lain. Yang baru di sini hanyalah bahasanya. Karena itu pertemuan pertama tidak membahas arsitektur atau basis data sama sekali dan dibatasi pada dua hal, yakni memastikan lingkungan kerja siap dan mengenali sintaks Go sampai cukup nyaman untuk membaca kode di pertemuan berikutnya.

Bahasa Go dirancang dengan sedikit fitur dan itu disengaja. Hampir tidak ada sihir seperti PHP di dalamnya. Tidak ada autoload, tidak ada container ajaib, tidak ada *exception* yang melompat entah ke mana. Semua yang terjadi terlihat di layar. Minggu pertama biasanya terasa ribet dan kompleks, setelahnya justru terasa lebih jujur dan fleksibel.

2. Variabel dan Konstanta

Go menyediakan beberapa cara mendeklarasikan variabel. Semuanya sah, dan masing-masing digunakan pada kondisi yang berbeda.

```
// 1. Tipe ditulis eksplisit
var nama string = "John Doe"
var umur int = 25

// 2. Tipe disimpulkan sendiri oleh Go
var kota = "Jakarta"

// 3. Deklarasi singkat – hanya boleh di dalam function
alamat := "Jl. Merdeka No. 1"
hobi := []string{"membaca", "coding"}

// 4. Deklarasi berkelompok
var (
    firstName string = "Jane"
    age        int    = 30
)

// 5. Zero value – nilai bawaan bila tidak diisi
var teksKosong string // ""
var angkaNol int      // 0
var salah bool        // false
```

Konsep zero value penting: di Go tidak ada variabel yang berisi "belum ada nilai". Setiap tipe punya nilai awal yang pasti. Ini menghilangkan sebagian besar kejutan yang biasa muncul dari null.

Konstanta dideklarasikan dengan `const` dan nilainya tidak dapat diubah:

```
const (
    StatusActive   = "ACTIVE"
    StatusInactive = "INACTIVE"
    MaxRetries     = 3
)
```

Eksplorasi sendiri

- ? Apa yang terjadi bila Anda menulis `alamat := "x"` di luar function? Coba, lalu baca pesan kesalahannya.
- ? Deklarasikan variabel lalu jangan gunakan sama sekali. Mengapa Go menolaknya, sementara bahasa lain hanya memperingatkan?
- ? Cari tahu apa itu `iota` dan untuk apa biasanya dipakai bersama `const`.

3. Struktur Data

Array dan slice

Array berukuran tetap dan jarang dipakai langsung. Slice berukuran dinamis, dan inilah yang anda gunakan hampir sepanjang waktu.

```
// Array – ukuran tetap, ikut menjadi bagian dari tipenya
var numbers [5]int = [5]int{1, 2, 3, 4, 5}
colors := [...]string{"red", "green", "blue"} // ukuran dihitung otomatis

// Slice – ukuran dinamis
var nilai []int
nilai = append(nilai, 10, 20, 30)

scores := make([]int, 5, 10) // panjang 5, kapasitas 10

// Mengiris slice
slice1 := []int{1, 2, 3, 4, 5}
slice2 := slice1[1:4] // [2 3 4]
slice3 := slice1[:3] // [1 2 3]
slice4 := slice1[2:] // [3 4 5]
```

Map

```
// Map harus diinisialisasi sebelum diisi
userAges := make(map[string]int)
userAges["John"] = 25

// Atau langsung dengan isinya
userRoles := map[string]string{
    "admin": "Administrator",
    "user": "Regular User",
}

// Pola dua nilai: hasilnya, dan apakah kuncinya ada
age, exists := userAges["John"]
if exists {
    fmt.Println(age)
}

delete(userAges, "John")

for key, value := range userRoles {
    fmt.Printf("%s: %s\n", key, value)
}
```

Channel

Channel adalah saluran untuk mengirim data ketika menggunakan goroutine. Di pertemuan ini cukup kenali bentuknya, pemakaian seriusnya menunggu pertemuan tentang *concurrency*.

```
ch := make(chan string) // tanpa buffer
buf := make(chan int, 3) // dengan buffer berkapasitas 3

go func() {
    ch <- "halo dari goroutine" // kirim
}()

pesan := <-ch // terima
fmt.Println(pesan)
```

Eksplorasi sendiri

- ? Apa bedanya panjang (len) dan kapasitas (cap) pada slice? Cetak keduanya sebelum dan sesudah append berkali-kali, lalu perhatikan polanya.
- ? Ubah satu elemen slice2 pada contoh di atas, lalu cetak slice1. Apa yang terjadi, dan mengapa?
- ? Coba tulis ke map yang dideklarasikan dengan var m map[string]int tanpa make. Baca pesan kesalahannya.
- ? Apa yang terjadi bila Anda mengirim ke channel tanpa buffer, tetapi tidak ada yang menerimanya?

4. Pointer

Pointer menyimpan alamat sebuah nilai, bukan salinannya. Analogi sederhana: variabel biasa itu fotokopi dokumen, pointer itu alamat lemari tempat dokumen aslinya disimpan.

```
x := 42
var p *int = &x    // & mengambil alamat

fmt.Println(x)      // 42
fmt.Println(*p)     // 42 - * membaca isi di alamat itu

*p = 100
fmt.Println(x)      // 100 - yang asli ikut berubah
```

Perbedaannya paling terasa ketika nilai dikirim ke sebuah function:

```
func ubahNilai(x int) {    // menerima SALINAN
    x = 100
}

func ubahLewatPointer(x *int) { // menerima ALAMAT
    *x = 100
}

num := 42
ubahNilai(num)           // num tetap 42
ubahLewatPointer(&num)   // num menjadi 100
```

Eksplorasi sendiri

- ? Slice dan map sudah berperilaku seperti reference meski dikirim tanpa &. Buktikan dengan sebuah function yang mengubah isinya. Lalu cari tahu mengapa demikian.
- ? Bila slice sudah berperilaku demikian, mengapa function tambahItem(s []*string, item string) tetap memerlukan pointer? Petunjuk: perhatikan apa yang dikembalikan oleh append.

5. Struct dan Method

Go tidak mengenal class. Data disimpan dalam struct, dan perilaku ditempelkan sebagai method pada struct tersebut.

```
type User struct {
    ID          int        `json:"id"`
    Username    string    `json:"username"`
    Email       string    `json:"email"`
    Password    string    `json:"-"` // tidak ikut menjadi JSON
    CreatedAt   time.Time `json:"created_at"`
    IsActive    bool      `json:"is_active"`
}

// (u *User) disebut receiver - inilah cara menempelkan method
func (u *User) Activate() { u.IsActive = true }
func (u *User) Deactivate() { u.IsActive = false }
```

```
func (u User) Info() string {
    return fmt.Sprintf("%s aktif: %v", u.Username, u.IsActive)
}
```

Value receiver atau pointer receiver? Bila ditulis (`u User`), method bekerja pada salinan dan perubahannya tidak tersimpan. Bila ditulis (`u *User`), method bekerja pada data aslinya. Gunakan pointer receiver bila method mengubah isi struct.

Struct tag dan embedded struct

Bagian di dalam tanda petik miring disebut struct tag. Tag inilah yang menentukan nama field ketika struct diubah menjadi JSON. Tag `json:"-"` membuat field tidak pernah ikut terkirim (seringkali berguna untuk password).

```
type Address struct {
    Street string `json:"street"`
    City    string `json:"city"`
}

type UserProfile struct {
    User      // embedded – field User ikut "naik" ke UserProfile
    Address   // embedded
    Bio string `json:"bio"`
}

profile := UserProfile{User: user1, Address: alamat, Bio: "Developer"}
fmt.Println(profile.Username, profile.City) // langsung, tanpa profile.User.Username
```

Eksplorasi sendiri

- ? Hapus tag json pada satu field, lalu kirim struct itu sebagai JSON. Nama field-nya berubah menjadi apa?
- ? Apa yang terjadi bila User dan Address sama-sama punya field bernama City? Coba, lalu baca pesan kesalahannya.
- ? Cari tahu perbedaan `json:"email,omitempty"` dengan `json:"email"` ketika nilainya kosong.

6. Fiber Sekilas

Fiber adalah framework web untuk Go dengan gaya penulisan yang mirip Express di Node.js. Tugasnya memangkas pekerjaan berulang: routing, konversi JSON, pengelompokan endpoint, dan middleware.

```
app := fiber.New()

app.Get("/", func(c *fiber.Ctx) error {
    return c.SendString("Hello, World!")
})

app.Listen(":3000")
```

Yang sering dipakai	Kegunaan
<code>c.Params("id")</code>	Mengambil bagian dinamis dari URL, misalnya <code>/users/:id</code>
<code>c.Query("nama")</code>	Mengambil parameter setelah tanda tanya pada URL (query params)
<code>c.BodyParser(&req)</code>	Mengubah JSON kiriman klien menjadi struct Go
<code>c.JSON(...)</code>	Mengirim respons berbentuk JSON
<code>c.Status(404)</code>	Menentukan status HTTP respons
<code>app.Group("/api/v1")</code>	Mengelompokkan route dengan awalan yang sama

Catatan versi. Mata kuliah ini memakai Fiber v2. Fiber v3 sudah ada dan sebagian API-nya berbeda, sehingga contoh kode v3 tidak akan dapat dikompilasi di sini. Saat mencari contoh di internet, pastikan yang Anda salin adalah versi 2.

B. Langkah Panduan

Panduan ini hanya membawa Anda sampai ada sesuatu yang berjalan. Setelah itu, silakan bongkar dan eksplorasi sendiri.

LANGKAH 1 — Instalasi dan Verifikasi

Pasang seluruh *tools* berikut. Beberapa di antaranya baru dipakai pada pertemuan berikutnya, tetapi lebih baik disiapkan sekarang sekalian.

Perangkat lunak	Sumber	Dipakai mulai
Go (versi stabil terbaru)	go.dev/dl	Pertemuan 1
Visual Studio Code + ekstensi Go	code.visualstudio.com	Pertemuan 1
Git + akun GitHub	git-scm.com dan github.com	Pertemuan 1
Postman atau Insomnia	postman.com	Pertemuan 1
PostgreSQL	postgresql.org/download	Pertemuan 3
MongoDB (lokal atau MongoDB Atlas)	mongodb.com	Pertemuan 10

Verifikasi lewat terminal. Semuanya harus mengembalikan nomor versi, bukan pesan kesalahan:

```
go version
git --version
psql --version
mongod --version
```

Ambil tangkapan layar setiap hasil verifikasi. Berkas-berkas tersebut menjadi bagian pertama tugas Anda.

LANGKAH 2 — Inisialisasi Project

```
mkdir latihan-fiber
cd latihan-fiber
go mod init latihan-fiber
go get github.com/gofiber/fiber/v2
```

Perintah `go mod init` membuat berkas `go.mod` yang mencatat nama modul dan daftar dependensi. Perannya serupa `composer.json` atau `package.json`. Berkas `go.sum` yang muncul kemudian berisi sidik jari setiap dependensi untuk memastikan isinya tidak berubah diam-diam.

Eksplorasi sendiri

- ? Buka berkas `go.mod` dan `go.sum`. Apa isi masing-masing, dan mengapa keduanya perlu ikut di-commit ke Git?
- ? Jalankan `go mod tidy`. Apa yang dikerjakannya, dan kapan Anda perlu menjalankannya?

LANGKAH 3 — Hello World dengan Fiber**► main.go**

```
package main

import (
    "log"

    "github.com/gofiber/fiber/v2"
)

func main() {
    app := fiber.New()

    app.Get("/", func(c *fiber.Ctx) error {
        return c.SendString("Hello, World!")
    })

    log.Fatal(app.Listen(":3000"))
}
```

```
go run main.go
```

Buka <http://localhost:3000> di browser. Bila muncul tulisan Hello, World!, lingkungan kerja anda sudah benar. Ambil tangkapan layarnya.

Eksplorasi sendiri

- ? Ubah port menjadi 8080. Apa saja yang perlu diubah, dan bagaimana cara mengujinya?
- ? Jalankan program kedua di port yang sama sementara yang pertama masih hidup. Baca pesan kesalahannya, lalu cari cara menghentikan proses yang menahan port tersebut.

LANGKAH 4 — Latihan Sintaks dalam Satu Berkas

Sebelum menyentuh API, biasakan dulu sintaksnya. Buat folder terpisah agar tidak bercampur dengan proyek Fiber anda.

```
mkdir ../latihan-sintaks && cd ../latihan-sintaks
go mod init latihan-sintaks
```

► main.go

```
package main

import "fmt"

type User struct {
    ID          int    `json:"id"`
    Username    string `json:"username"`
    IsActive    bool   `json:"is_active"`
}

func (u *User) Activate() { u.IsActive = true }
func (u User) Info() string { return fmt.Sprintf("%s aktif: %v", u.Username, u.IsActive) }

func tukarNilai(a, b *int) { *a, *b = *b, *a }
```



```

func main() {
    // Variabel
    var nama string = "Sari"
    umur := 20
    var kosong string
    fmt.Printf("%s %d zero value: %q\n", nama, umur, kosong)

    // Slice
    nilai := []int{10, 20, 30}
    nilai = append(nilai, 40)
    fmt.Println(nilai, nilai[1:3], len(nilai), cap(nilai))

    // Map
    skor := make(map[string]int)
    skor["Sari"] = 90
    if n, ada := skor["Budi"]; ada {
        fmt.Println("Budi:", n)
    } else {
        fmt.Println("Budi belum punya nilai")
    }

    // Channel
    ch := make(chan string)
    go func() { ch <- "halo dari goroutine" }()
    fmt.Println(<-ch)

    // Pointer
    a, b := 1, 2
    tukarNilai(&a, &b)
    fmt.Println("setelah ditukar:", a, b)

    // Struct
    u := User{ID: 1, Username: "sari"}
    u.Activate()
    fmt.Println(u.Info())
}

```

```
go run main.go
```

Rusak dengan sengaja

Hapus salah satu baris `fmt.Println`, lalu jalankan ulang. Kompilasi akan gagal dengan pesan "imported and not used". Kembalikan lagi.

Di Go, membaca pesan kesalahan kompilasi adalah kegiatan sehari-hari, bukan keadaan darurat. Semakin cepat anda terbiasa, semakin ringan sisa semester ini.

C. Tugas Mandiri

Tugas dikerjakan sendiri. Anda boleh mencari referensi dari mana pun, tetapi harus mampu menjelaskan setiap baris kode yang anda kumpulkan.

1. Bukti Persiapan Lingkungan

Sertakan tangkapan layar untuk:

- Hasil `go version`, `git --version`, `psql --version`, dan `mongod --version`
- Visual Studio Code dengan ekstensi Go sudah terpasang
- Terminal setelah menjalankan `go mod init`
- Tampilan Hello, World! di browser

2. Variabel dan Struktur Data

Buat satu program Go yang mendemonstrasikan:

- Deklarasi lima variabel dengan tipe berbeda: `string`, `int`, `float64`, `bool`, dan `slice`
- Sebuah map untuk menyimpan data mahasiswa, dengan nama sebagai kunci dan nilai sebagai isinya
- Operasi pada map: menambah, membaca dengan pengecekan keberadaan, menghapus, dan menelusuri seluruh isinya

3. Pointer

Buat program berisi dua function berikut, lalu buktikan cara kerjanya:

Function	Perilaku yang diharapkan
<code>swap(a, b *int)</code>	Menukar nilai dua integer melalui pointer
<code>updateSlice(s []*string, newItem string)</code>	Menambahkan item baru ke slice

Sertakan pula perbandingan pass by value dengan pass by pointer beserta keluaran programnya, dan jelaskan singkat mengapa hasilnya berbeda.

4. Struct Student

Buat struct Student dengan field ID, Name, Grade, dan IsActive, beserta method:

Method	Kegunaan
<code>GetInfo() string</code>	Mengembalikan informasi lengkap student dalam satu baris teks
<code>UpdateGrade(grade float64)</code>	Memperbarui nilai
<code>Activate()</code> dan <code>Deactivate()</code>	Mengubah status aktif

Tentukan sendiri mana method yang memakai value receiver dan mana yang memakai pointer receiver, lalu jelaskan alasan pilihan anda dalam laporan.

Ketentuan Pengumpulan

1. Laporan dikumpulkan dalam bentuk PDF dengan nama berkas Tugas1_NIM.pdf
2. Laporan memuat: tangkapan layar, potongan kode penting, keluaran program, dan penjelasan singkat untuk setiap bagian
3. Seluruh kode diunggah ke repositori GitHub, dan tautannya dicantumkan pada halaman pertama laporan
4. Buatlah commit yang bertahap. Satu commit berisi seluruh pekerjaan tidak diterima
5. Cantumkan pula sumber bantuan yang anda gunakan, termasuk alat bantu AI, beserta bagian mana yang dibantu

Rujukan

- A Tour of Go — go.dev/tour
- Effective Go — go.dev/doc/effective_go
- Dokumentasi Fiber v2 — docs.gofiber.io
- Status HTTP — developer.mozilla.org/en-US/docs/Web/HTTP/Status